Assignment 2(1)

Priority Scheduler

20215174 임혁준

20225087 성준호

Submit : 2025.04.14

1. 문제 분석

기존의 PintOS의 scheduling은 FIFO 방식으로 구현된 ready list를 사용한 Round-Robin 형식의 scheduling이다. 이는 thread의 priority를 고려하지 않은 scheduling 방식으로, CPU를 사용할 중요한 작업과 덜 중요한 작업의 우선순위를 구분하지 못하기에 마우스 입력, 화면 렌더링처럼 실시간 반응이 필요한 작업에 비효율적이다. 또한 lock이나 리소스를 가진 thread가 느린 thread라면 그것을 기다리는 다른 빠른 thread도 묶여버릴 수 있다는 문제점이 존재한다. 이를 해결하기 위한 scheduling 방식으로 Priority Scheduling이 존재하며, Problem 1은 해당 scheduler를 구현하는 것이다. 이를 구현하기 위해서는 ready list에 thread가 priority에 따라 sort된 채로 존재하도록 해야 하며, ready list에 변화가 생길 때마다 priority scheduling을 재실행해야 한다. 또한 priority donation을 구현해야 하는데, 이는 thread가 lock을 기다릴 때마다, lock을 점유하고 있는 thread에게 자신의 priority를 상속하는 것이다. 이를 통해 중간 priority의 thread가 더 높은 priority의 thread보다 먼저 가장 낮은 priority의 thread의 lock을 점유하는 상황인 priority inversion을 방지할 수 있다. 따라서 synch.c의 lock에 priority donation 기능을 추가하여야 한다.

2. 알고리즘 설계

ready_list에 thread가 priority에 따라 정렬되어 있을 수 있도록, list_push_back() 대신 list_insert_ordered()를 사용해 ready_list에 thread가 정렬된 상태로 insert 되도록 한다. list_insert_ordered() 함수의 사용을 위해 priority의 크기를 비교하는 비교함수 또한 정의되어야 한다. 우선순위가 변동이 생길 때마다 priority scheduling을 재실행하기 위한 함수 역시 정의하고 적절하게 사용되어야 한다.

1. thread를 priority에 따라 정렬하기 위한 비교함수 compare_priority() 선언
2. 우선순위 변동 시 thread_yield()를 호출하는 함수 thread_check_preemption() 선언
3. thread_unblock, thread_yield 함수 수정: list_push_back() 대신 list_insert_ordered() 사용
4. thread_create 후 thread_check_preemption() 호출
5. thread_set_priority 후, thread_check_preemption() 호출

sema 역시 priority에 따라 정렬되어 있을 수 있도록 함수를 수정해야 한다.

6. sema_down 함수 수정: list_push_back() 대신 list_insert_ordered 사용
7. sema_up 함수 수정: thread_unblock() 전 list_sort() 사용, 후 thread_check_preemption() 호출
8. sema를 priority에 따라 정렬하기 위한 비교함수 compare_sema_priority() 선언
9. cond_wait 함수 수정: list_push_back() 대신 list_insert_ordered() 사용
10. cond_signal 함수 수정: sema_up() 전 list_sort() 사용

이후, 우리는 priority donation을 구현해야 한다. priority donation 구현을 위해 thread struct에 donation 이전 원래의 priority 값을 저장하는 변수와 기부목록을 저장하는 배열을 선언한다.

11. thread struct 에 donation 을 위한 변수 선언 및 초기화

이에 따라 thread_set_priority 함수가 추가적으로 수정된다.

12. thread_set_priority에 donation 기능 추가

기부가 발생할 때마다, thread의 priority는 기부된 목록 중 최대값으로 갱신되어야 한다. 이를 위해 기부목록에서 priority의 크기를 비교하는 함수를 선언하여야 한다.

13. donations를 priority로 정렬하기 위한 비교함수 compare_donation 선언

lock이 걸리고 해제될 때마다 우선순위를 기부하는 기능의 추가를 위해, lock_acquire와 lock_release를 수정한다.

14. 우선순위 donation하는 함수 priority_donation() 선언

15. lock_acquire에 donation 기능 추가

16. lock_release에 donation 기능 추가

최종적으로, 우리는 다음의 절차를 진행해야 한다.

1. thread를 priority에 따라 정렬하기 위한 비교함수 compare_priority() 선언

2. 우선순위 변동 시 thread_yield()를 호출하는 함수 thread_check_preemption() 선언

3. thread_unblock, thread_yield 함수 수정: list_push_back() 대신 list_insert_ordered() 사용

4. thread_create 후 thread_check_preemption() 호출

5. thread_set_priority 후, thread_check_preemption() 호출

6. sema_down 함수 수정: list_push_back() 대신 list_insert_ordered 사용

7. sema_up 함수 수정: thread_unblock() 전 list_sort() 사용, 후 thread_check_preemption() 호출

8. sema를 priority에 따라 정렬하기 위한 비교함수 compare_sema_priority() 선언

9. cond_wait 함수 수정: list_push_back() 대신 list_insert_ordered() 사용

10. cond_signal 함수 수정: sema_up() 전 list_sort() 사용

11. thread struct 에 donation 을 위한 변수 선언 및 초기화

12. thread_set_priority에 donation 기능 추가

13. donations를 priority로 정렬하기 위한 비교함수 compare_donation 선언

14. 우선순위 donation하는 함수 priority_donation() 선언

15. lock_acquire에 donation 기능 추가

16. lock_release에 donation 기능 추가

3. 구현

1. thread t를 priority로 정렬하기 위한 함수 compare_priority() 선언

```c
/* Additional functions for priroty scheduling. */
bool
compare_priority(struct list_elem* a, struct list_elem* b, void* aux UNUSED)
{
    if (list_entry(a, struct thread, elem) -> priority > list_entry(b, struct thread, elem) -> priority)
        return true;
    return false;
}
```

thread.c

```c
/* Additional functions for priroty schedulling. */
bool compare_priority(struct list_elem* a, struct list_elem* b, void* aux);
```

thread.h

2. 우선순위 변동 시 thread_yield()를 호출하는 함수 thread_check_preemption() 선언

```c
void
thread_check_preemption(void)
{
    if (!list_empty(&ready_list) &&
        thread_current()->priority < list_entry(list_front(&ready_list), struct thread, elem)->priority)
        thread_yield();
}
```

thread.c

```c
void thread_check_preemption(void);
```

thread.h

3. thread_unblock, thread_yield 함수 수정: list_push_back() 대신 list_insert_ordered() 사용

```
void
thread_yield (void)
{
  struct thread *cur = thread_current ();
  enum intr_level old_level;

  ASSERT (!intr_context ());

  old_level = intr_disable ();
  if (cur != idle_thread)
    list_insert_ordered(&ready_list, &cur->elem, compare_priority, NULL);
    //list_push_back (&ready_list, &cur->elem);
  cur->status = THREAD_READY;
  schedule ();
  intr_set_level (old_level);
}
```

thread.c

```
void
thread_unblock (struct thread *t)
{
  enum intr_level old_level;

  ASSERT (is_thread (t));

  old_level = intr_disable ();
  ASSERT (t->status == THREAD_BLOCKED);

  list_insert_ordered(&ready_list, &t->elem, compare_priority, NULL);
  //list_push_back (&ready_list, &t->elem);
  t->status = THREAD_READY;
  intr_set_level (old_level);
}
```

thread.c

4. thread_create 후 thread_check_preemption() 호출

```
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
  struct thread *t;
```

...

```
  intr_set_level (old_level);

  /* Add to run queue. */
  thread_unblock (t);
  thread_check_preemption();

  return tid;
}
```

thread.c

5. thread_set_priority 후 thread_check_prreemption() 호출

```c
/* Sets the current thread's priority to NEW_PRIORITY. */
void
thread_set_priority (int new_priority)
{
    if (thread_mlfqs) return;
    struct thread* cur = thread_current();

    cur->priority_backup = new_priority;
    cur->priority = new_priority;
    if (!list_empty(&cur->donations))
    {
        struct thread* thread_top = list_entry(list_front(&cur->donations), struct thread, donation_elem);
        if (cur->priority < thread_top->priority)
            cur->priority = thread_top->priority;
    }

    thread_check_preemption();
}
```

thread.c

6. sema_down 함수 수정: list_push_back() 대신 list_insert_ordered 사용

```c
void
sema_down (struct semaphore *sema)
{
  enum intr_level old_level;

  ASSERT (sema != NULL);
  ASSERT (!intr_context ());

  old_level = intr_disable ();
  while (sema->value == 0)
    {
      list_insert_ordered(&sema->waiters, &thread_current()->elem, compare_priority, NULL);
      //list_push_back (&sema->waiters, &thread_current ()->elem);
      thread_block ();
    }
  sema->value--;
  intr_set_level (old_level);
}
```

synch.c

7. sema_up 함수 수정: thread_unblock() 전 list_sort() 사용, 후 thread_check_preemption() 호출

```c
void
sema_up (struct semaphore *sema)
{
  enum intr_level old_level;

  ASSERT (sema != NULL);

  old_level = intr_disable ();
  if (!list_empty(&sema->waiters))
  {
      list_sort(&sema->waiters, compare_priority, NULL);
      thread_unblock(list_entry(list_pop_front(&sema->waiters), struct thread, elem));
  }
  sema->value++;
  thread_check_preemption();
  intr_set_level (old_level);
}
```

synch.c

8. sema를 priority에 따라 정렬하기 위한 비교함수 compare_sema_priority() 선언

```c
bool
compare_sema_priority(const struct list_elem* a, const struct list_elem* b, void* aux UNUSED)
{
    struct semaphore_elem* sema_a = list_entry(a, struct semaphore_elem, elem);
    struct semaphore_elem* sema_b = list_entry(b, struct semaphore_elem, elem);
    struct thread* thread_a = list_entry(
        list_begin(&(sema_a->semaphore.waiters)), struct thread, elem);
    struct thread* thread_b = list_entry(
        list_begin(&(sema_b->semaphore.waiters)), struct thread, elem);

    return thread_a->priority > thread_b->priority;
}
```

synch.c

```c
/* Additional functions for priority scheduling and donation. */
bool compare_sema_priority(const struct list_elem* a, const struct list_elem* b, void* aux);
```

synch.h

9. cond_wait 함수 수정: list_push_back() 대신 list_insert_ordered() 사용

```
void
cond_wait (struct condition *cond, struct lock *lock)
{
  struct semaphore_elem waiter;

  ASSERT (cond != NULL);
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (lock_held_by_current_thread (lock));

  sema_init (&waiter.semaphore, 0);
  list_insert_ordered(&cond->waiters, &waiter.elem, compare_sema_priority, NULL);
  //list_push_back (&cond->waiters, &waiter.elem);
  lock_release (lock);
  sema_down (&waiter.semaphore);
  lock_acquire (lock);
}
```

synch.c

10. cond_signal 함수 수정: sema_up() 전 list_sort() 사용

```
void
cond_signal (struct condition *cond, struct lock *lock UNUSED)
{
  ASSERT (cond != NULL);
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (lock_held_by_current_thread (lock));

  if (!list_empty(&cond->waiters))
  {
      list_sort(&cond->waiters, compare_sema_priority, NULL);
      sema_up(&list_entry(list_pop_front(&cond->waiters),
          struct semaphore_elem, elem)->semaphore);
  }
}
```

synch.c

11. thread struct 에 donation 을 위한 변수 선언 및 초기화

```c
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list. */


    /* Variable for avoiding busy-waiting. */
    int64_t wake_time;

    /* Variables for mlfqs. */
    int nice;
    int recent_cpu;

    /* Variables for priority donation. */
    int priority_backup;
    struct list donations;
    struct list_elem donation_elem;
    struct lock* waiting_lock;
```

thread.h

```c
static void
init_thread (struct thread *t, const char *name, int priority)
{
  ASSERT (t != NULL);
  ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
  ASSERT (name != NULL);

  memset (t, 0, sizeof *t);
  t->status = THREAD_BLOCKED;
  strlcpy (t->name, name, sizeof t->name);
  t->stack = (uint8_t *) t + PGSIZE;
  t->priority = priority;
  t->magic = THREAD_MAGIC;
  list_push_back (&all_list, &t->allelem);

  /* Added for priority donation. */
  t->priority_backup = priority;
  list_init(&t->donations);
  t->waiting_lock = NULL;
```

thread.c

12. thread_set_priority 에 donation 기능 추가

```c
/* Sets the current thread's priority to NEW_PRIORITY. */
void
thread_set_priority (int new_priority)
{
    if (thread_mlfqs) return;
    struct thread* cur = thread_current();

    cur->priority_backup = new_priority;
    cur->priority = new_priority;
    if (!list_empty(&cur->donations))
    {
        struct thread* thread_top = list_entry(list_front(&cur->donations), struct thread, donation_elem);
        if (cur->priority < thread_top->priority)
            cur->priority = thread_top->priority;
    }

    thread_check_preemption();
}
```

thread.c

13. donations 를 priority 로 정렬하기 위한 비교함수 compare_donation 선언

```c
/* Additional functions for priority scheduling and donation. */
bool
compare_donation(const struct list_elem* a, const struct list_elem* b, void* aux UNUSED)
{
    return list_entry(a, struct thread, donation_elem)->priority >
        list_entry(b, struct thread, donation_elem)->priority;
}
```

synch.c

```c
bool compare_donation(const struct list_elem* a, const struct list_elem* b, void* aux);
```

synch.h

14. 우선순위 donation 하는 함수 priority_donation() 선언

```c
void
priority_donation()
{
    struct thread* cur = thread_current();
    int i = 0;
    for (i = 0; i < 8; i += 1)  // MAX_DEPTH = 8
    {
        if (!cur->waiting_lock)
            return;
        cur->waiting_lock->holder->priority = cur->priority;
        cur = cur->waiting_lock->holder;
    }
}
```

synch.c

```c
void priority_donation();
```

synch.h

15.lock_acquire 에 donation 기능 추가

```
void
lock_acquire (struct lock *lock)
{
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (!lock_held_by_current_thread (lock));

  struct thread* cur = thread_current();
  if (lock->holder != NULL)
  {
      cur->waiting_lock = lock;
      list_insert_ordered(&lock->holder->donations, &cur->donation_elem, compare_donation, NULL);
      if (!thread_mlfqs) priority_donation();
  }
  sema_down(&lock->semaphore);
  cur->waiting_lock = NULL;
  lock->holder = cur;
}
```

synch.c

16. lock_release 에 donation 기능 추가

```
void
lock_release (struct lock *lock)
{
    ASSERT(lock != NULL);
    ASSERT(lock_held_by_current_thread(lock));

    struct thread* cur = thread_current();

    if (!thread_mlfqs)
    {
        struct list_elem* i;
        for (i = list_begin(&cur->donations); i != list_end(&cur->donations); i = list_next(i))
        {
            struct thread* thread_i = list_entry(i, struct thread, donation_elem);
            if (thread_i->waiting_lock == lock)
                list_remove(&thread_i->donation_elem);
        }

        cur->priority = cur->priority_backup;
        if (!list_empty(&cur->donations))
        {
            list_sort(&cur->donations, compare_donation, NULL);
            struct thread* thread_top = list_entry(list_front(&cur->donations), struct thread, donation_elem);
            if (cur->priority < thread_top->priority)
                cur->priority = thread_top->priority;
        }
    }

    lock->holder = NULL;
    sema_up(&lock->semaphore);
}
```

synch.c

4. 결과

- make test 실행결과

```
hjabo@Pintos:~/pintos/src/threads$ make
cd build && make all
make[1]: Entering directory `/home/hjabo/pintos/src/threads/build'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/hjabo/pintos/src/threads/build'
hjabo@Pintos:~/pintos/src/threads$ date
2025. 04. 13. (일) 17:52:05 KST
hjabo@Pintos:~/pintos/src/threads$ make check -s
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
```

실행 결과에 따르면, alarm과 priority 테스트를 모두 성공적으로 pass하였다.